BiteFixes Enterprise Architecture Manual

Volume 14

Canonical Data Model Specification (CDMS)

Enterprise Edition

Document ID: **BFA-014**

Version: **2.0**

Status: **Architecture Baseline**

Table of Contents

Chapter 1

Introduction

The Canonical Data Model is the official representation of all business information inside BiteFixes.

Every API

Every workflow

Every AI interaction

Every report

Every integration

must use this model.

The database is only one implementation.

Chapter 2

Domain First

The architecture follows Domain Driven Design.

The order is:

```
Business

↓

Domain

↓

Entities

↓

Relationships

↓

Services

↓

Database

↓

API
```

Never the opposite.

Chapter 3

Core Domains

The entire platform is divided into independent business domains.

```
Identity

Company
```

```
Customer

Device

Service

Ticket

Inventory

Knowledge

Workflow

Billing

Notification

AI

Analytics

Audit
```

Each domain owns its own entities.

Chapter 4

Aggregate Roots

Every domain has one Aggregate Root.

Example

```
Company

↓
```

Company Aggregate

Customer Domain

↓

Customer Aggregate

Ticket Domain

↓

Ticket Aggregate

Inventory Domain

↓

Inventory Aggregate

No entity outside the aggregate modifies internal state directly.

Chapter 5

Canonical Entity Rules

Every entity follows the same structure.

Example

UUID

Created At

Updated At

Created By

Updated By

Version

Status

Company ID

Metadata

Every entity.

No exceptions.

Chapter 6

Universal Fields

Every table contains

id

created_at

updated_at

created_by

updated_by

version

status

company_id

Additionally

deleted_at

deleted_by

for Soft Delete.

Chapter 7

Entity Naming

Always singular.

Correct

`Customer`

`Ticket`

`Device`

`Company`

Database implementation

`customers`

`tickets`

`devices`

`companies`

Chapter 8

Value Objects

Instead of duplicating fields

Create reusable Value Objects.

Example

Address

Street

City

State

Country

ZipCode

Phone

CountryCode

AreaCode

Number

Extension

Money

Currency

Amount

Language

Code

Locale

Timezone

Chapter 9

Relationships

Prefer references over duplication.

Example

```
Customer

↓

Device

↓

Ticket
```

Never duplicate Customer Name inside Ticket.

Reference Customer.

Chapter 10

Multi Tenant

Every business entity belongs to one Company.

Example

```
Company

↓

Customers

↓
```

```
Tickets

↓

Knowledge

↓

Inventory

↓

Reports
```

Only Platform entities are global.

Chapter 11

Audit

Every modification produces an Audit Event.

Audit is not mixed with operational data.

Separate domain.

Chapter 12

Soft Delete

Nothing important is physically deleted.

Lifecycle

```
Active
↓
Inactive
↓
Archived
↓
Deleted
```

Deleted means

Hidden.

Not removed.

Chapter 13

Versioning

Business entities support optimistic locking.

Every modification increments

`version`

This avoids concurrent updates.

Chapter 14

Metadata

Instead of constantly modifying schema

Entities support metadata.

Example

```
{
  "preferredColor":"blue",
  "favoriteBrand":"Dell"
}
```

Metadata never replaces core fields.

Chapter 15

Domain Ownership

Customer Domain

owns

`Customer`

`CustomerAddress`

`CustomerPreference`

Ticket Domain

owns

`Ticket`

`TicketStatus`

`TicketHistory`

`TicketAttachment`

Inventory Domain

owns

`Part`

`Warehouse`

`Stock`

`Reservation`

No cross ownership.

Chapter 16

Cross Domain Communication

Domains communicate only by

Events

or

Application Services.

Never

Database Updates.

Chapter 17

Database Independence

The Canonical Model supports

PostgreSQL

MySQL

SQL Server

Oracle

MongoDB

Future databases

without changing the Domain Model.

Chapter 18

Evolution Rules

Schema evolution follows:

Add

↓

Deprecate

↓

Migrate

↓

Remove

Never

Rename directly.

Never

Delete directly.

Chapter 19

Canonical Hierarchy

```
Platform
|
├── Company
|
├── Identity
|
├── Customer
|
├── Device
|
├── Ticket
|
```

```
├── Service
│
├── Inventory
│
├── Billing
│
├── Workflow
│
├── Knowledge
│
├── AI
│
├── Notification
│
└── Audit
```

Chapter 20

The Next Evolution

Aquí es donde considero que BiteFixes puede diferenciarse claramente de otros SaaS.

En lugar de pasar directamente al diseño físico de PostgreSQL, propondría introducir una capa adicional:

```
Business

↓

Canonical Domain Model

↓

Canonical Data Model

↓

Logical Data Model

↓

Physical Database Model

↓

Supabase/PostgreSQL
```

¿Por qué?

La mayoría de los proyectos pasan directamente de las entidades del negocio a las tablas de la base de datos. Eso hace que, con el tiempo, el diseño termine condicionado por PostgreSQL o por el ORM.

Con esta capa intermedia:

- El **Canonical Domain Model** define el negocio.
- El **Canonical Data Model** define la información.
- El **Logical Data Model** organiza esa información en entidades y relaciones independientes del motor.
- El **Physical Database Model** adapta el modelo lógico a PostgreSQL (hoy Supabase, mañana podría ser otro).